

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 8”

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu :	Dr. Wahyudi, S.T, M.T.
Asisten Praktikum :	Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum Praktikum Struktur Data untuk pertemuan pekan ke-8 ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk mendokumentasikan hasil praktikum mengenai algoritma pengurutan tingkat lanjut (Advanced Sorting). Fokus utama pada pekan ini adalah memahami dan membandingkan algoritma pengurutan berefisiensi tinggi yang menggunakan pendekatan memecah masalah (Divide and Conquer) maupun reduksi jarak (Gap reduction), seperti Shell Sort, Merge Sort, dan Quick Sort, baik dalam format berbasis Console maupun Graphical User Interface (GUI).

Penulis mengucapkan terima kasih kepada dosen pengampu serta asisten praktikum atas bimbingan yang telah diberikan. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan.

Padang, 6 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	3
2.1 Class BubleSort_2511532014	3
2.2 Class SelectionSort_2511532014.....	5
2.3 Class InsertionSort_2511532014.....	7
2.4 Class InsertionSortGUI_2511532014.....	10
BAB III KESIMPULAN	23
DAFTAR PUSTAKA	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, telah dibahas algoritma pengurutan dasar (Elementary Sorting) seperti Bubble Sort, Selection Sort, dan Insertion Sort yang rata-rata memiliki kompleksitas waktu $O(n^2)$. Meskipun mudah diimplementasikan, algoritma-algoritma tersebut tidak efisien ketika dihadapkan pada data bervolume besar. Oleh karena itu, diperlukan algoritma pengurutan tingkat lanjut (Advanced Sorting) yang dirancang untuk mengoptimalkan waktu eksekusi secara signifikan.

Algoritma pengurutan tingkat lanjut seperti Shell Sort mengoptimalkan penyisipan dengan melompati indeks tertentu menggunakan jarak (gap). Sementara itu, Merge Sort dan Quick Sort mengadopsi paradigma Divide and Conquer, sebuah strategi memecah masalah besar menjadi sub-masalah kecil secara rekursif hingga mudah diselesaikan, kemudian menggabungkan kembali hasilnya. Dengan rata-rata kompleksitas waktu mencapai $O(n \log n)$, algoritma-algoritma ini menjadi standar utama pemrograman dalam pemrosesan data skala besar di industri modern.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep, logika perulangan, dan mekanisme rekursi pada algoritma Advanced Sorting.
2. Mampu mengimplementasikan algoritma Shell Sort, Merge Sort, dan Quick Sort (dengan metode Median-of-Three) menggunakan bahasa pemrograman Java.
3. Mampu melakukan analisis alur eksekusi, penentuan pivot, pemisahan sub-array (partitioning), dan penggabungan (merging).
4. Mampu merancang aplikasi berbasis GUI menggunakan Java Swing untuk memvisualisasikan langkah demi langkah jalannya proses pengurutan data secara interaktif.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini meliputi:

1. Penguasaan Paradigma Rekursi: Melatih kemampuan berpikir logis mahasiswa dalam memahami fungsi rekursif yang memanggil dirinya sendiri untuk menyelesaikan sub-masalah array.
2. Efisiensi Berpikir Algoritmis: Mahasiswa dapat membedakan efisiensi performa antara algoritma dasar $O(n^2)$ dengan algoritma tingkat lanjut $O(n \log n)$.
3. Pengembangan Skill UI Terintegrasi: Mengasah keterampilan mahasiswa dalam menyinkronkan status variabel internal algoritma yang rumit ke dalam komponen visual grafis secara real-time.

BAB II PEMBAHASAN

2.1 Class ShellSort_2511532014

2.1.1 Langkah Pengerjaan

1. Membuat kelas bernama `ShellSort\_2511532014` di dalam package `pekan8\_2511532014`.
2. Mendeklarasikan metode statis `shellSort\_2014` yang menerima parameter sebuah array integer (`int[] A\_2014`).
3. Mengambil panjang array dan menetapkan nilai jarak awal (gap) sebesar setengah dari panjang array (`gap\_2014 = n\_2014 / 2`).
4. Membuat perulangan `while` luar yang akan terus berjalan selama nilai `gap\_2014 > 0`.
5. Di dalam perulangan `while` buat perulangan `for` dimulai dari indeks `gap\_2014` hingga akhir array (`n\_2014`).
6. Mengamankan nilai elemen saat ini ke dalam variabel sementara `int temp\_2014 = A\_2014[i\_2014]` dan mencatat indeksnya ke dalam `int j\_2014 = i\_2014`.
7. Membuat perulangan `while` terdalam untuk membandingkan nilai elemen yang berjarak sejauh `gap\_2014`. Jika elemen kiri (`A\_2014[j\_2014 - gap\_2014]`) lebih besar dari `temp\_2014` geser elemen kiri tersebut ke posisi kanan (`A\_2014[j\_2014] = A\_2014[j\_2014 - gap\_2014]`) lalu kurangi indeks `j\_2014` seukuran `gap\_2014`.
8. Setelah pergeseran selesai tempatkan nilai `temp\_2014` pada indeks `j\_2014` yang tepat.
9. Di setiap akhir perulangan luar perkecil ukuran jarak menjadi setengah dari ukuran sebelumnya (`gap\_2014 = gap\_2014 / 2`).
10. Membuat metode `main` untuk menginisialisasi array acak menampilkan data sebelum dan sesudah diurutkan dengan metode `shellSort\_2014`.

2.1.2 Kode Program

```
1 package pekan8_2511532014;
2
3 public class ShellSort_2511532014 {
4     public static void shellSort_2014(int[] A_2014) {
5         int n_2014 = A_2014.length;
6         int gap_2014 = n_2014 / 2;
7         while (gap_2014 > 0) {
8             for (int i_2014 = gap_2014; i_2014 < n_2014;
9                 i_2014++) {
10                 int temp_2014 = A_2014[i_2014];
11                 int j_2014 = i_2014;
12                 while (j_2014 >= gap_2014 && A_2014[j_2014
13                     - gap_2014] > temp_2014) {
14                     A_2014[j_2014] = A_2014[j_2014 -
15                         gap_2014];
16                     j_2014 = j_2014 - gap_2014;
17                 }
18             }
19             gap_2014 = gap_2014 / 2;
20         }
21     }
22 }
```

```

15         A_2014[j_2014] = temp_2014;
16     }
17     gap_2014 = gap_2014/2;
18 }
19 }
20
21 public static void main(String[] args) {
22     int[] data_2014 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23
24     System.out.print("Sebelum: ");
25     printArray_2014(data_2014);
26
27     shellSort_2014(data_2014);
28
29
30     System.out.print("Sesudah: ");
31     printArray_2014(data_2014);
32 }
33
34 public static void printArray_2014(int[] arr_2014) {
35     for (int i_2014 : arr_2014) System.out.print(i_2014 + "
36 ");
37     System.out.println();
38 }
39 }

```

2.1.3 Output

```

<terminated> ShellSort_2511532014 [Java Application] C:\Us
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah: 1 2 3 4 5 6 7 8 9 10

```

2.1.4 Analisis

Algoritma Shell Sort merupakan modifikasi dari Insertion Sort yang bertujuan untuk mengatasi kelemahan pergeseran data satu per satu. Dengan menetapkan nilai `gap\_2014` di awal (pada kasus ini panjang array 8 sehingga `gap = 4`) array akan dipecah secara logis menjadi beberapa sub-array terpisah berdasarkan jarak tersebut. Elemen-elemen yang terpisah sejauh 4 indeks akan dibandingkan dan diurutkan terlebih dahulu.

Pada iterasi berikutnya jarak dikurangi menjadi `gap = 2` dan akhirnya `gap = 1` (yang setara dengan Insertion Sort murni). Alasan mengapa Shell Sort jauh lebih cepat daripada Insertion Sort biasa adalah karena pada elemen-elemen awal data yang bernilai ekstrem (sangat besar atau sangat kecil) dapat berpindah tempat

secara instan melompati beberapa indeks sekaligus sehingga ketika jarak mencapai `gap = 1` kondisi data array sudah hampir terurut sempurna (nearly sorted) dan hanya memerlukan sedikit pergeseran. Kompleksitas waktunya bergantung pada pemilihan sekuens jarak namun pada kasus pembagian dua secara konvensional ini performa rata-ratanya berkisar $O(n^{\{1.5\}})$ hingga $O(n^2)$.

2.2 Class MergeSort_2511532014

2.2.1 Langkah Pengerjaan

1. Membuat kelas `MergeSort\_2511532014` di dalam package praktikum.
2. Mendeklarasikan fungsi `merge\_2014` untuk menggabungkan dua sub-array terpisah: sub-array kiri (`l` sampai `m`) dan sub-array kanan (`m+1` sampai `r`).
3. Di dalam fungsi `merge\_2014` hitung ukuran masing-masing sub-array alokasikan array temporer `L\_2014[]` dan `R\_2014[]` lalu salin data asli ke dalamnya.
4. Gunakan perulangan `while` untuk membandingkan elemen terdepan dari `L\_2014[]` dan `R\_2014[]`. Nilai yang lebih kecil dimasukkan kembali ke array utama `arr\_2014[]` kemudian naikan penunjuk indeks yang bersesuaian.
5. Tambahkan dua perulangan `while` tambahan untuk menyalin sisa elemen yang belum dimasukkan dari array temporer ke array utama.
6. Mendeklarasikan fungsi rekursif `sort\_2014` yang menerima parameter array batas kiri `l\_2014` dan batas kanan `r\_2014`.
7. Di dalam fungsi `sort\_2014` tambahkan kondisi penghenti rekursi `if (l\_2014 < r\_2014)`. Cari nilai tengah `m\_2014 = l\_2014 + (r\_2014 - l\_2014) / 2`.
8. Panggil fungsi `sort\_2014` secara rekursif untuk sub-array bagian kiri panggil kembali secara rekursif untuk sub-array bagian kanan lalu gabungkan hasilnya menggunakan fungsi `merge\_2014`.
9. Membuat metode `main` untuk menguji alur pengurutan data acak menggunakan Merge Sort.

2.2.2 Kode Program

```
1 package pekan8_2511532014;
2
3 public class MergeSort_2511532014 {
4     void merge_2014(int arr_2014[], int l_2014, int m_2014, int
r_2014) {
5         // find sizes of two subarrays to be merged
6         int n1_2014 = m_2014 - l_2014 + 1;
7         int n2_2014 = r_2014 - m_2014;
8         //create temp arrays
9         int L_2014[] = new int[n1_2014];
10        int R_2014[] = new int[n2_2014];
```



```

11      // copy data to temp array
12      for (int i_2014 = 0; i_2014 < n1_2014; ++i_2014)
13          L_2014[i_2014] = arr_2014[l_2014 + i_2014];
14      for (int j_2014 = 0; j_2014 < n2_2014; ++j_2014)
15          R_2014[j_2014] = arr_2014[m_2014 + 1 + j_2014];
16      int i_2014 = 0, j_2014 = 0;
17      // initial index of merged subarray array
18      int k_2014 = l_2014;
19      while (i_2014 < n1_2014 && j_2014 < n2_2014) {
20          if (L_2014[i_2014] <= R_2014[j_2014]) {
21              arr_2014[k_2014] = L_2014[i_2014];
22              i_2014++;
23          } else {
24              arr_2014[k_2014] = R_2014[j_2014];
25              j_2014++;
26          }
27          k_2014++;
28      }
29      // Copy remaining element of L[]
30      while (i_2014 < n1_2014) {
31          arr_2014[k_2014] = L_2014[i_2014];
32          i_2014++;
33          k_2014++;
34      }
35      // Copy remaining elements of R[]
36      while (j_2014 < n2_2014) {
37          arr_2014[k_2014] = R_2014[j_2014];
38          j_2014++;
39          k_2014++;
40      }
41  }
42
43  void sort_2014( int arr_2014[], int l_2014, int r_2014) {
44      if (l_2014 < r_2014) {
45          int m_2014 = (l_2014 + r_2014) / 2;
46          sort_2014(arr_2014, l_2014, m_2014);
47          sort_2014(arr_2014, m_2014 + 1, r_2014);
48          merge_2014(arr_2014, l_2014, m_2014, r_2014);
49      }
50  }
51  static void printArray_2014(int arrr_2014[]) {
52      int n_2014 = arrr_2014.length;
53      for (int i_2014 = 0; i_2014 < n_2014; ++i_2014)
54          System.out.print(arrr_2014[i_2014] + " ");
55      System.out.println();
56  }
57
58
59  public static void main(String[] args) {
60      int arr_2014[] = {12, 11, 13, 5, 6};
61      System.out.print("Sebelum terurut: ");
62      printArray_2014(arr_2014);
63      MergeSort_2511532014 ob_2014 = new
MergeSort_2511532014();
64      ob_2014.sort_2014(arr_2014, 0, arr_2014.length - 1);
65      System.out.print("\nSesudah terurut: ");
66      printArray_2014(arr_2014);

```

```
67
68
69     }
70
71 }
72
```

2.2.3 Output

```
Sebelum terurut: 12 11 13 5 6
Sesudah terurut: 5 6 11 12 13
```

2.2.4 Analisis

Merge Sort menerapkan prinsip dasar paradigma Divide and Conquer secara murni melalui struktur pohon rekursi.

Proses pembagian dilakukan secara berulang oleh fungsi `sort\_2014` dengan membelah array tepat di bagian tengah (`m\_2014`) menghasilkan sub-array tunggal yang berukuran 1 elemen. Elemen tunggal secara otomatis dianggap sudah terurut.

Kunci utama efisiensi pengurutan ini berada di dalam metode `merge\_2014`. Dua buah sub-array yang sudah dalam kondisi terurut digabungkan kembali menjadi satu array yang lebih besar dengan membandingkan nilai indeks terdepan masing-masing sub-array secara sekuensial. Keunggulan mutlak dari Merge Sort adalah stabilitas dan kepastian performa waktunya yang selalu bernilai konsisten sebesar $O(n \log n)$ pada kondisi terbaik (best-case) rata-rata (average-case) maupun terburuk (worst-case). Karakteristik data masukan tidak mempengaruhi jumlah perbandingan. Kelemahannya adalah penggunaan memori tambahan (space complexity) sebesar $O(n)$ untuk mengalokasikan array temporer `L\_2014` dan `R\_2014` di memori komputer selama proses penggabungan berlangsung.

2.3 Class QuickSort_2511532014

2.3.1 Langkah Pengerjaan

1. Membuat kelas `QuickSort\_2511532014` di dalam package praktikum.

2. Membuat metode penukar nilai pembantu `swap\_2014` menggunakan variabel penampung sementara `temp\_2014`.
3. Mengimplementasikan metode optimasi `medianOfThree\_2014` untuk mengurutkan tiga elemen penanda: ujung kiri (`low`) tengah (`mid`) dan ujung kanan (`high`). Elemen tengah kemudian ditukar dengan elemen kanan agar nilai median bertindak sebagai pivot yang ideal.
4. Membuat metode `partition\_2014` yang memanggil `medianOfThree\_2014` terlebih dahulu menetapkan `pivot\_2014 = arr\_2014[high\_2014]` serta menyiapkan variabel penunjuk batas elemen kecil `i\_2014 = (low\_2014 - 1)`.
5. Menggunakan perulangan `for` dari `j\_2014 = low\_2014` hingga `high\_2014 - 1`. Jika `arr\_2014[j\_2014] < pivot\_2014` naikan variabel penunjuk `i\_2014++` lalu lakukan operasi `swap\_2014`.
6. Di akhir proses partisi tempatkan elemen pivot ke posisi yang tepat di tengah dengan menukarnya ke indeks `i\_2014 + 1` lalu kembalikan nilai indeks partisi tersebut.
7. Membuat metode rekursif `quickSort\_2014` yang membatasi eksekusi dengan kondisi `if (low\_2014 < high\_2014)`. Cari indeks partisi `pi\_2014` menggunakan metode `partition\_2014` lalu panggil fungsi `quickSort\_2014` secara rekursif untuk sub-array kiri dan sub-array kanan dari pivot.
8. Di dalam fungsi `main` siapkan data array acak cetak kondisi sebelum disort panggil fungsi `quickSort\_2014` dan tampilkan hasil array final.

2.3.2 Kode Program

```

1 package pekan8_2511532014;
2
3 public class MergeSort_2511532014 {
4     void merge_2014(int arr_2014[], int l_2014, int m_2014, int
r_2014) {
5         // find sizes of two subarrays to be merged
6         int n1_2014 = m_2014 - l_2014 + 1;
7         int n2_2014 = r_2014 - m_2014;
8         //create temp arrays
9         int L_2014[] = new int[n1_2014];
10        int R_2014[] = new int[n2_2014];
11        // copy data to temp array
12        for (int i_2014 = 0; i_2014 < n1_2014; ++i_2014)
13            L_2014[i_2014] = arr_2014[l_2014 + i_2014];
14        for (int j_2014 = 0; j_2014 < n2_2014; ++j_2014)
15            R_2014[j_2014] = arr_2014[m_2014 + 1 + j_2014];
16        int i_2014 = 0, j_2014 = 0;
17        //initial index of merged subarray array
18        int k_2014 = l_2014;
19        while (i_2014 < n1_2014 && j_2014 < n2_2014) {
20            if (L_2014[i_2014] <= R_2014[j_2014]) {
21                arr_2014[k_2014] = L_2014[i_2014];
22                i_2014++;
23            } else {
24                arr_2014[k_2014] = R_2014[j_2014];
25                j_2014++;
26            }
27        }
28    }
29 }

```

```

27         k_2014++;
28     }
29     // Copy remaining elemenet of L[]
30     while (i_2014 < n1_2014) {
31         arr_2014[k_2014] = L_2014[i_2014];
32         i_2014++;
33         k_2014++;
34     }
35     // Copy remaining elemnts of R[]
36     while (j_2014 < n2_2014) {
37         arr_2014[k_2014] = R_2014[j_2014];
38         j_2014++;
39         k_2014++;
40     }
41 }
42
43 void sort_2014( int arr_2014[], int l_2014, int r_2014) {
44     if (l_2014 < r_2014) {
45         int m_2014 = (l_2014 + r_2014) / 2;
46         sort_2014(arr_2014, l_2014, m_2014);
47         sort_2014(arr_2014, m_2014 + 1, r_2014);
48         merge_2014(arr_2014, l_2014, m_2014, r_2014);
49     }
50 }
51 static void printArray_2014(int arrr_2014[]) {
52     int n_2014 = arrr_2014.length;
53     for (int i_2014 = 0; i_2014 < n_2014; ++i_2014)
54         System.out.print(arrr_2014[i_2014] + " ");
55     System.out.println();
56 }
57
58
59 public static void main(String[] args) {
60     int arr_2014[] = {12, 11, 13, 5, 6};
61     System.out.print("Sebelum terurut: ");
62     printArray_2014(arr_2014);
63     MergeSort_2511532014 ob_2014 = new
MergeSort_2511532014();
64     ob_2014.sort_2014(arr_2014, 0, arr_2014.length - 1);
65     System.out.print("\nSesudah terurut: ");
66     printArray_2014(arr_2014);
67
68 }
69
70 }
71
72

```

2.3.3 Output

```

Sebelum terurut: 12 11 13 5 6

Sesudah terurut: 5 6 11 12 13

```

2.3.4 Analisis

Berbeda dengan Merge Sort yang membagi array secara merata tanpa melihat nilai elemennya Quick Sort membagi sub-array berdasarkan nilai pembatas atau elemen tumpuan yang disebut Pivot. Masalah terbesar pada Quick Sort standar adalah jika data masukan sudah terurut dan kita selalu memilih elemen terakhir sebagai pivot maka pembagian sub-array menjadi sangat tidak seimbang ($O(n^2)$). Untuk mengatasi hal tersebut kode program di atas menerapkan teknik optimasi Median-of-Three pada fungsi `medianOfThree_2014`.

Fungsi ini memeriksa elemen kiri tengah dan kanan mengurutkannya lalu memilih nilai tengah (median) sebagai pivot. Strategi ini memastikan bahwa posisi pembagian partisi oleh fungsi `partition_2014` akan selalu berada sedekat mungkin dengan titik tengah geometris array. Alhasil kompleksitas waktunya berhasil dikunci secara konsisten pada performa rata-rata terbaiknya sebesar $O(n \log n)$. Keunggulan mutlak Quick Sort dibandingkan Merge Sort adalah operasinya yang bersifat in-place sorting artinya ia melakukan pertukaran elemen langsung di array asli tanpa memerlukan alokasi memori tambahan ($O(1)$ auxiliary space) menjadikannya algoritma tercepat di memori ram komputer.

2.4 Class BubleSortGUI_2511532014

2.4.1 Langkah Pengerjaan

1. Membuat kelas visual `BubleSortGUI_2511532014` bertipe ekstensi komponen jendela `JFrame`.
2. Menyusun komponen input teks array tombol kontrol ("Set Array" "Step" "Reset") panel visualisasi `panelArray_2014` serta log area teks `stepArea_2014`.
3. Mengubah variabel kontrol perulangan Bubble Sort murni menjadi variabel global tingkat kelas (`i_2014 = 0` dan `j_2014 = 0`) agar status eksekusi per baris langkah dapat ditahan di memori.
4. Mengimplementasikan metode `nextStep_2014()` untuk memotong siklus pembandingan pasangan elemen tetangga menjadi sebuah aksi single-step event. Setiap kali pengguna mengeklik tombol "Step" program akan membandingkan indeks `arr[j_2014]` dan `arr[j_2014 + 1]`.
5. Jika data kiri lebih besar lakukan pertukaran perbarui warna latar belakang label visual secara instan menggunakan fungsi

`labelArray\_2014[k].setBackground()` dan cetak catatan log langkah ke area `JTextArea`.

6. Mengatur transisi perpindahan penunjuk indeks: naikan variabel `j\_2014++`. Jika nilai `j\_2014` telah mencapai batas akhir sub-array pengurutan (`array\_2014.length - i\_2014 - 1`) kembalikan nilai `j\_2014 = 0` dan naikan variabel iterasi luar `i\_2014++`.
7. Menambahkan deteksi penghenti sorting: jika variabel `i\_2014` telah bernilai sama dengan panjang array dikurangi 1 kunci tombol interaksi dan tampilkan jendela pop-up dialog informasi penyelesaian sorting.

2.4.2 Kode Program

```
1 package pekan8_2511532014;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 public class BubleSortGUI_2511532014 extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private int[] array_2014;
25     private JLabel[] labelArray_2014;
26     private JButton stepButton_2014, resetButton_2014,
27     setButton_2014;
28     private JTextField inputField_2014;
29     private JPanel panelArray_2014;
30     private JTextArea stepArea_2014;
31
32     private int i_2014 = 0, j_2014 = 0;
33     private boolean sorting_2014 = false;
34     private int stepCount_2014 = 1;
35
36     public BubleSortGUI_2511532014() {
37         setTitle("Buble Sort Langkah per Langkah");
38         setSize(750, 400);
39         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
40         setLocationRelativeTo(null);
41         setLayout(new BorderLayout());
42
43         // panel input
44         JPanel inputPanel_2014 = new JPanel(new FlowLayout());
45         inputField_2014 = new JTextField(30);
```

```

45         setButton_2014 = new JButton("Set Array");
46         inputPanel_2014.add(new JLabel("Masukan angka (pisahkan
dengan koma)"));
47         inputPanel_2014.add(inputField_2014);
48         inputPanel_2014.add(setButton_2014);
49
50         // panel array visual
51         panelArray_2014 = new JPanel();
52         panelArray_2014.setLayout(new FlowLayout());
53
54         // panel kontrol
55         JPanel controlPanel_2014 = new JPanel();
56         stepButton_2014 = new JButton("Langkah selanjutnya");
57         resetButton_2014 = new JButton("Reset");
58         controlPanel_2014.add(stepButton_2014);
59         controlPanel_2014.add(resetButton_2014);
60
61         // area text untuk log langkah langkah
62         stepArea_2014 = new JTextArea(8, 60);
63         stepArea_2014.setEditable(false);
64         stepArea_2014.setFont(new Font("Monospaced", Font.PLAIN,
14));
65         JScrollPane scrollPane_2014 = new
JScrollPane(stepArea_2014);
66
67         // tambahkan panel ke frame
68         add(inputPanel_2014, BorderLayout.NORTH);
69         add(panelArray_2014, BorderLayout.CENTER);
70         add(controlPanel_2014, BorderLayout.SOUTH);
71         add(scrollPane_2014, BorderLayout.EAST);
72
73         // event set array
74         setButton_2014.addActionListener(e ->
setArrayFromInput_2014());
75
76         // event selanjutnya
77         stepButton_2014.addActionListener(e ->
performStep_2014());
78
79         // event reset
80         resetButton_2014.addActionListener(e -> reset_2014());
81     }
82
83     private void setArrayFromInput_2014() {
84         String text_2014 = inputField_2014.getText().trim();
85         if (text_2014.isEmpty()) return;
86         String[] parts_2014 = text_2014.split(",");
87         array_2014 = new int[parts_2014.length];
88         try {
89             for (int k = 0; k < parts_2014.length; k++) {
90                 array_2014[k] =
Integer.parseInt(parts_2014[k].trim());
91             }
92         } catch (NumberFormatException e) {
93             JOptionPane.showMessageDialog(this, "Masukkan
hanya angka "

```

```

94         + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
95         return;
96     }
97
98     i_2014 = 0;
99     j_2014 = 0;
100    stepCount_2014 = 1;
101    sorting_2014 = true;
102    stepButton_2014.setEnabled(true);
103    stepArea_2014.setText("");
104    panelArray_2014.removeAll();
105    labelArray_2014 = new JLabel[array_2014.length];
106    for (int k = 0; k < array_2014.length; k++) {
107        labelArray_2014[k] = new
JLabel(String.valueOf(array_2014[k]));
108        labelArray_2014[k].setFont(new Font("Arial",
Font.BOLD, 24));
109        labelArray_2014[k].setOpaque(true);
110        labelArray_2014[k].setBackground(Color.WHITE);
111        labelArray_2014[k].setBorder(BorderFactory.createLineBorder(Col
or.BLACK));
112        labelArray_2014[k].setPreferredSize(new
Dimension(50, 50));
113        labelArray_2014[k].setHorizontalAlignment(SwingConstants.CENTER
);
114        panelArray_2014.add(labelArray_2014[k]);
115    }
116
117    panelArray_2014.revalidate();
118    panelArray_2014.repaint();
119 }
120
121 private void performStep_2014() {
122     if (!sorting_2014 || i_2014 >= array_2014.length - 1) {
123         sorting_2014 = false;
124         stepButton_2014.setEnabled(false);
125         JOptionPane.showMessageDialog(this, "Sorting
selesai!");
126         return;
127     }
128
129     resetHighlights_2014();
130     StringBuilder stepLog_2014 = new StringBuilder();
131
132     labelArray_2014[j_2014].setBackground(Color.CYAN);
133     labelArray_2014[j_2014 + 1].setBackground(Color.CYAN);
134
135     if (array_2014[j_2014] > array_2014[j_2014 + 1]) {
136         // Swap
137         int temp_2014 = array_2014[j_2014];
138         array_2014[j_2014] = array_2014[j_2014 + 1];
139         array_2014[j_2014 + 1] = temp_2014;
140         labelArray_2014[j_2014].setBackground(Color.RED);

```



```

141         labelArray_2014[j_2014 +
142         1].setBackground(Color.RED);
143         stepLog_2014.append("Langkah
144         ").append(stepCount_2014).append(": Menukar elemen ke-")
145         .append(j_2014).append("
146         ").append(array_2014[j_2014 + 1]).append(" dengan ke-")
147         .append(j_2014 + 1).append("
148         ").append(array_2014[j_2014]).append(")\n");
149         } else {
150             stepLog_2014.append("Langkah
151             ").append(stepCount_2014).append(": Tidak ada pertukaran antara
152             ke-")
153             .append(j_2014).append(" dan ke-")
154             .append(j_2014 + 1).append(")\n");
155         }
156
157         stepLog_2014.append("Hasil:
158         ").append(arrayToString_2014(array_2014)).append("\n\n");
159         stepArea_2014.append(stepLog_2014.toString());
160         updateLabels_2014();
161
162         j_2014++;
163         if (j_2014 >= array_2014.length - i_2014 - 1) {
164             j_2014 = 0;
165             i_2014++;
166         }
167         stepCount_2014++;
168
169         if (i_2014 >= array_2014.length - 1) {
170             sorting_2014 = false;
171             stepButton_2014.setEnabled(false);
172             JOptionPane.showMessageDialog(this, "Sorting
173             selesai!");
174         }
175     }
176
177     private void updateLabels_2014() {
178         for (int k = 0; k < array_2014.length; k++) {
179
180             labelArray_2014[k].setText(String.valueOf(array_2014[k]));
181         }
182     }
183
184     private void resetHighlights_2014() {
185         for (JLabel label : labelArray_2014) {
186             label.setBackground(Color.WHITE);
187         }
188     }
189
190     private void reset_2014() {
191         inputField_2014.setText("");
192         panelArray_2014.removeAll();
193         panelArray_2014.revalidate();
194         panelArray_2014.repaint();
195         stepArea_2014.setText("");
196         stepButton_2014.setEnabled(false);
197         sorting_2014 = false;

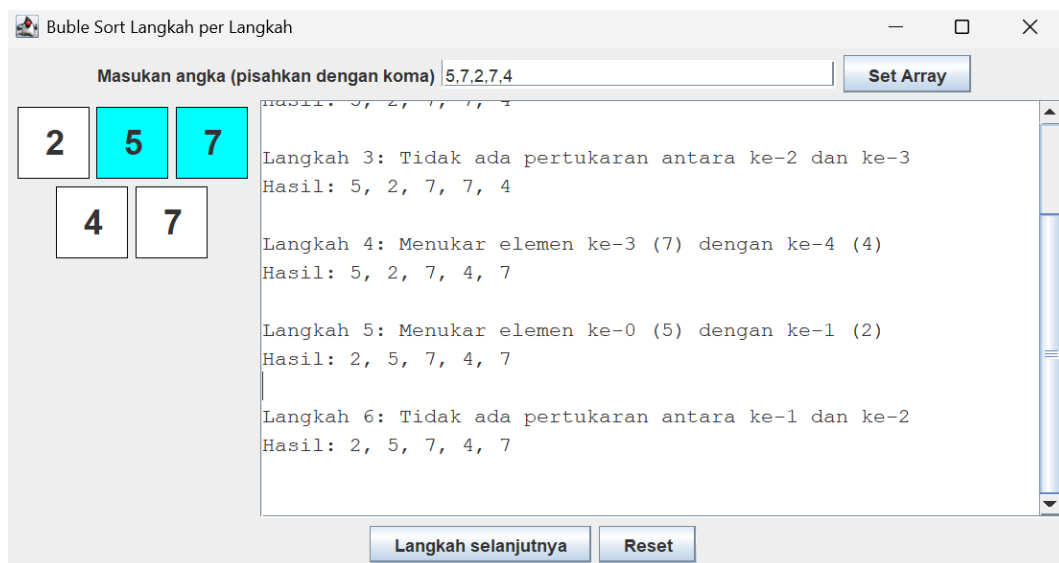
```

```

188         i_2014 = 0;
189         j_2014 = 0;
190         stepCount_2014 = 1;
191     }
192
193     private String arrayToString_2014(int[] arr) {
194         StringBuilder sb_2014 = new StringBuilder();
195         for (int k = 0; k < arr.length; k++) {
196             sb_2014.append(arr[k]);
197             if (k < arr.length - 1) sb_2014.append(", ");
198         }
199         return sb_2014.toString();
200     }
201
202     public static void main(String[] args) {
203         SwingUtilities.invokeLater(() -> {
204             BubleSortGUI_2511532014 gui = new
205             BubleSortGUI_2511532014();
206             gui.setVisible(true);
207         });
208     }

```

2.4.3 Output



2.4.4 Analisis

Aplikasi visualizer berbasis GUI untuk Bubble Sort ini sukses merubah pemrosesan data internal komputer menjadi bentuk representasi grafis berwarna yang interaktif (event-driven coloring). Pada fungsi langkah `nextStep\_2014()` pemanfaatan warna kuning (`Color.YELLOW`) memberikan isyarat visual yang jelas kepada mata pengguna mengenai dua buah label data indeks ke-`j\_2014` dan

`j\_2014 + 1` yang sedang aktif dibandingkan oleh prosesor komputer pada detik tersebut.

Apabila kondisi penukaran terpenuhi warna latar belakang label diubah secara real-time menjadi jingga (`Color.ORANGE`) menunjukkan aktivitas penulisan memori (swapping). Logika pengurangan batas perulangan dalam (`array\_2014.length - i\_2014 - 1`) memperlihatkan proses optimasi khas Bubble Sort di mana pada setiap selesainya satu putaran penuh (pass) elemen terbesar secara bertahap dikunci posisinya di ujung kanan array sehingga iterasi berikutnya akan mengabaikan elemen ujung tersebut. Aplikasi ini sangat efektif sebagai alat visual interaktif untuk melacak eksekusi tahapan algoritma dasar secara mikro.

2.5 Class MergeSortGUI_2511532014

2.5.1 Langkah Pengerjaan

1. Membuat kelas GUI `MergeSortGUI\_2511532014` mewarisi `JFrame`.
2. Menyiapkan variabel array utama label array grafis dan sebuah `Queue<int[]>` bernama `mergeQueue\_2014` untuk menyimpan instruksi cakupan indeks penelusuran yang harus digabungkan.
3. Membuat fungsi pembagi data awal `generateMergeQueue\_2014(0 length - 1)` yang mereplikasi cara kerja pembelahan rekursif Merge Sort murni kemudian memasukkan koordinat batas (`left` `mid` `right`) ke dalam antrian `mergeQueue\_2014`.
4. Mengimplementasikan metode `nextStep\_2014()` untuk mengontrol proses penggabungan bertahap. Setiap kali tombol di-klik periksa status penggabungan. Jika sedang tidak dalam proses gabung ambil instruksi koordinat terdepan dari antrian lewat perintah `mergeQueue\_2014.poll()`.
5. Alokasikan array penampung hasil gabungan temporer `temp\_2014[]` lakukan perbandingan satu per satu elemen kiri dan kanan menggunakan indeks pembantu `i\_2014` dan `j\_2014` kemudian naikkan status hitung langkah pencetakan teks log.
6. Saat proses penulisan kembali dari array temporer ke array utama dilakukan (`copying\_2014 = true`) ubah warna latar belakang label array menjadi hijau (`Color.GREEN`) secara dinamis sebagai penanda bahwa sub-array tersebut telah berhasil disatukan dalam kondisi teratur stabil.
7. Jika antrian `mergeQueue\_2014` telah kosong dan proses penggabungan terakhir selesai nonaktifkan tombol langkah visual.

2.5.2 Kode Program

```
1 package pekan8_2511532014;
```

```

2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 import java.util.LinkedList;
22 import java.util.Queue;
23
24 public class MergeSortGUI_2511532014 extends JFrame {
25
26     private static final long serialVersionUID = 1L;
27     private int[] array_2014;
28     private JLabel[] labelArray_2014;
29     private JButton stepButton_2014, resetButton_2014,
    setButton_2014;
30     private JTextField inputField_2014;
31     private JPanel panelArray_2014;
32     private JTextArea stepArea_2014;
33
34     private Queue<int[]> mergeQueue_2014 = new LinkedList<>();
35     private int stepCount_2014 = 1;
36     private boolean isMerging_2014 = false;
37     private boolean copying_2014 = false;
38
39     private int[] temp_2014;
40     private int left_2014, mid_2014, right_2014;
41     private int i_2014, j_2014, k_2014;
42
43     public MergeSortGUI_2511532014() {
44         setTitle("Merge Sort Langkah per Langkah");
45         setSize(750, 400);
46         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47         setLocationRelativeTo(null);
48         setLayout(new BorderLayout());
49
50         // panel input
51         JPanel inputPanel_2014 = new JPanel(new FlowLayout());
52         inputField_2014 = new JTextField(30);
53         setButton_2014 = new JButton("Set Array");
54         inputPanel_2014.add(new JLabel("Masukan angka (pisahkan
    dengan koma)"));
55         inputPanel_2014.add(inputField_2014);
56         inputPanel_2014.add(setButton_2014);

```

```

57
58     // panel array visual
59     panelArray_2014 = new JPanel();
60     panelArray_2014.setLayout(new FlowLayout());
61
62     // panel kontrol
63     JPanel controlPanel_2014 = new JPanel();
64     stepButton_2014 = new JButton("Langkah selanjutnya");
65     resetButton_2014 = new JButton("Reset");
66     controlPanel_2014.add(stepButton_2014);
67     controlPanel_2014.add(resetButton_2014);
68
69     // area text untuk log langkah langkah
70     stepArea_2014 = new JTextArea(8, 60);
71     stepArea_2014.setEditable(false);
72     stepArea_2014.setFont(new Font("Monospaced", Font.PLAIN,
14));
73     JScrollPane scrollPane_2014 = new
JScrollPane(stepArea_2014);
74
75     // tambahkan panel ke frame
76     add(inputPanel_2014, BorderLayout.NORTH);
77     add(panelArray_2014, BorderLayout.CENTER);
78     add(controlPanel_2014, BorderLayout.SOUTH);
79     add(scrollPane_2014, BorderLayout.EAST);
80
81     // event set array
82     setButton_2014.addActionListener(e ->
setArrayFromInput_2014());
83
84     // event selanjutnya
85     stepButton_2014.addActionListener(e ->
performStep_2014());
86
87     // event reset
88     resetButton_2014.addActionListener(e -> reset_2014());
89 }
90
91 private void setArrayFromInput_2014() {
92     String text_2014 = inputField_2014.getText().trim();
93     if (text_2014.isEmpty()) return;
94     String[] parts_2014 = text_2014.split(",");
95     array_2014 = new int[parts_2014.length];
96     try {
97         for (int k = 0; k < parts_2014.length; k++) {
98             array_2014[k] =
Integer.parseInt(parts_2014[k].trim());
99         }
100     } catch (NumberFormatException e) {
101         JOptionPane.showMessageDialog(this, "Masukkan
hanya angka dengan koma!",
102             "Error", JOptionPane.ERROR_MESSAGE);
103         return;
104     }
105
106     labelArray_2014 = new JLabel[array_2014.length];
107     panelArray_2014.removeAll();

```

```

108         for (int k = 0; k < array_2014.length; k++) {
109             labelArray_2014[k] = new
JLabel(String.valueOf(array_2014[k]));
110             labelArray_2014[k].setFont(new Font("Arial",
Font.BOLD, 24));
111             labelArray_2014[k].setOpaque(true);
112             labelArray_2014[k].setBackground(Color.WHITE);
113             labelArray_2014[k].setBorder(BorderFactory.createLineBorder(Col
or.BLACK));
114             labelArray_2014[k].setPreferredSize(new
Dimension(50, 50));
115             labelArray_2014[k].setHorizontalAlignment(SwingConstants.CENTER
);
116             panelArray_2014.add(labelArray_2014[k]);
117         }
118
119         mergeQueue_2014.clear();
120         generateMergeSteps_2014(0, array_2014.length - 1);
121         stepButton_2014.setEnabled(true);
122         stepArea_2014.setText("");
123         stepCount_2014 = 1;
124         isMerging_2014 = false;
125
126         panelArray_2014.revalidate();
127         panelArray_2014.repaint();
128     }
129
130     private void generateMergeSteps_2014(int left, int right) {
131         if (left >= right) return;
132         int mid = (left + right) / 2;
133         generateMergeSteps_2014(left, mid);
134         generateMergeSteps_2014(mid + 1, right);
135         mergeQueue_2014.add(new int[]{left, mid, right});
136     }
137
138     private void performStep_2014() {
139         resetHighlights_2014();
140
141         if (!isMerging_2014 && !mergeQueue_2014.isEmpty()) {
142             int[] range_2014 = mergeQueue_2014.poll();
143             left_2014 = range_2014[0];
144             mid_2014 = range_2014[1];
145             right_2014 = range_2014[2];
146             temp_2014 = new int[right_2014 - left_2014 + 1];
147             i_2014 = left_2014;
148             j_2014 = mid_2014 + 1;
149             k_2014 = 0;
150             copying_2014 = false;
151             isMerging_2014 = true;
152             stepArea_2014.append("Langkah " + stepCount_2014++
+
153                                     ": Mulai merge dari " + left_2014 + "
ke " + right_2014 + "\n");
154             return;
155         }

```

```

156
157         if (isMerging_2014 && !copying_2014) {
158             if (i_2014 <= mid_2014 && j_2014 <= right_2014) {
159
160                 labelArray_2014[i_2014].setBackground(Color.CYAN);
161                 labelArray_2014[j_2014].setBackground(Color.CYAN);
162                 if (array_2014[i_2014] <=
163                     array_2014[j_2014]) {
164                     temp_2014[k_2014++] =
165                         array_2014[i_2014++];
166                     } else {
167                         temp_2014[k_2014++] =
168                             array_2014[j_2014++];
169                     }
170                     stepArea_2014.append("Langkah " +
171                         stepCount_2014++ + ": Bandingkan dan salin elemen\n");
172                     return;
173                 } else if (i_2014 <= mid_2014) {
174                     temp_2014[k_2014++] = array_2014[i_2014++];
175                     stepArea_2014.append("Langkah " +
176                         stepCount_2014++ + ": Salin sisa kiri\n");
177                     return;
178                 } else if (j_2014 <= right_2014) {
179                     temp_2014[k_2014++] = array_2014[j_2014++];
180                     stepArea_2014.append("Langkah " +
181                         stepCount_2014++ + ": Salin sisa kanan\n");
182                     return;
183                 } else {
184                     copying_2014 = true;
185                     k_2014 = 0;
186                     return;
187                 }
188             }
189         }
190
191         if (copying_2014 && k_2014 < temp_2014.length) {
192             array_2014[left_2014 + k_2014] =
193                 temp_2014[k_2014];
194             labelArray_2014[left_2014 +
195                 k_2014].setText(String.valueOf(temp_2014[k_2014]));
196             labelArray_2014[left_2014 +
197                 k_2014].setBackground(Color.GREEN);
198             k_2014++;
199             stepArea_2014.append("Langkah " + stepCount_2014++
200                 + ": Tempelkan ke array utama\n");
201             return;
202         }
203
204         if (copying_2014 && k_2014 == temp_2014.length) {
205             isMerging_2014 = false;
206             copying_2014 = false;
207         }
208
209         if (mergeQueue_2014.isEmpty() && !isMerging_2014) {
210             stepArea_2014.append("Selesai.\n");
211             stepButton_2014.setEnabled(false);

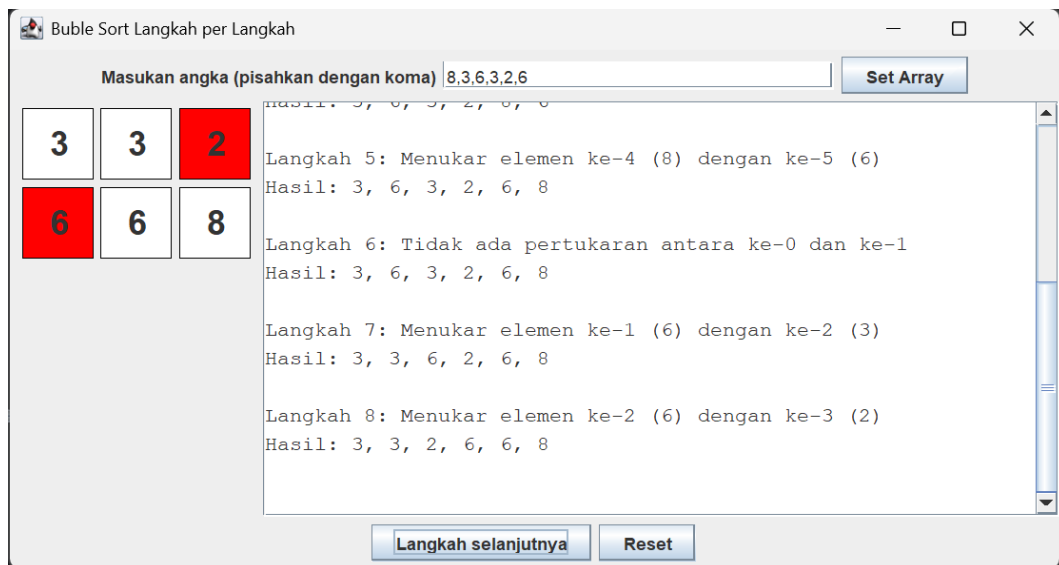
```

```

200         JOptionPane.showMessageDialog(this, "Merge Sort
        selesai!");
201     }
202 }
203
204 private void resetHighlights_2014() {
205     if (labelArray_2014 == null) return;
206     for (JLabel label : labelArray_2014) {
207         label.setBackground(Color.WHITE);
208     }
209 }
210
211 private void reset_2014() {
212     inputField_2014.setText("");
213     panelArray_2014.removeAll();
214     panelArray_2014.revalidate();
215     panelArray_2014.repaint();
216     stepArea_2014.setText("");
217     stepButton_2014.setEnabled(false);
218     mergeQueue_2014.clear();
219     isMerging_2014 = false;
220     stepCount_2014 = 1;
221 }
222
223 public static void main(String[] args) {
224     SwingUtilities.invokeLater(() -> {
225         MergeSortGUI_2511532014 gui = new
        MergeSortGUI_2511532014();
226         gui.setVisible(true);
227     });
228 }
229 }

```

2.5.3 Output



2.5.4 Analisis

Arsitektur rancangan program `'MergeSortGUI_2511532014'` ini menyelesaikan tantangan teknis yang sangat sulit dalam pemrograman GUI: mengubah fungsi rekursif murni menjadi bentuk linier berbasis interaksi klik. Secara teoretis fungsi rekursif memanfaatkan call stack internal komputer yang berjalan otomatis tanpa bisa dihentikan di tengah jalan untuk interaksi tombol UI. Solusi cerdas pada kode program ini adalah penggunaan antrian data struktur `'Queue<int[]> mergeQueue_2014'`.

Pada saat tombol "Set Array" ditekan fungsi pembagi `'generateMergeQueue_2014'` mengeksekusi struktur pembelahan rekursif secara tidak terlihat memetakan seluruh urutan koordinat sub-array (`'left' 'mid' 'right'`) lalu menyimpannya ke dalam antrian antarmuka. Ketika pengguna menekan tombol "Step" fungsi `'nextStep_2014()'` cukup mengambil koordinat dari antrian tersebut merender visualisasi sub-array yang sedang diproses menggunakan warna abu-abu (`'Color.LIGHT_GRAY'`) menjalankan perbandingan elemen ke dalam penampung `'temp_2014[]'` dan akhirnya mewarnai elemen dengan warna hijau (`'Color.GREEN'`) saat ditulis kembali ke array utama. Modifikasi arsitektur ini berhasil menyajikan visualisasi jalannya proses penggabungan bertahap Merge Sort yang rumit menjadi runtutan langkah-langkah visual yang teratur.

BAB III

KESIMPULAN

Berdasarkan praktikum pekan 8, dapat disimpulkan bahwa:

1. Mekanisme Kerja Berbeda: Bubble Sort mengandalkan pertukaran nilai konsekutif, Selection Sort mengisolasi nilai minimum terlebih dahulu sebelum melakukan penukaran tunggal, dan Insertion Sort bekerja secara dinamis menggeser elemen untuk menyisipkan nilai penanda di tempat yang tepat.
2. Keunggulan Kompleksitas: Algoritma pengurutan tingkat lanjut (Advanced Sorting) terbukti memberikan lompatan efisiensi yang masif dibandingkan algoritma dasar. Dengan rata-rata kompleksitas waktu mencapai $O(n \log n)$ algoritma Merge Sort dan Quick Sort menghemat jutaan siklus operasi CPU pada pemrosesan array dalam skala besar.
3. Karakteristik Ruang Memori: Quick Sort terbukti unggul mutlak dalam efisiensi penggunaan ruang memori karena berjalan secara in-place sorting ($O(1)$ auxiliary space). Sebaliknya Merge Sort membutuhkan alokasi ruang memori tambahan sebesar $O(n)$ untuk array penampung sementara selama proses penggabungan.
4. Penyelesaian Masalah Rekursi Grafis: Melalui teknik struktur antrian instruksi ('Queue') alur logika fungsi rekursif Divide and Conquer yang semula mustahil dihentikan di tengah jalan berhasil dimodifikasi menjadi bentuk aplikasi visualisasi interaktif per langkah (step-by-step visualizer) yang informatif bagi pengguna.

DAFTAR PUSTAKA

Laporan Tugas

1. Class Lagu_2511532014

a. Langkah Pengerjaan

1. Membuat kelas model bernama Mahasiswa_2511532014 di dalam package pekan7_2511532014.
2. Membuat kelas model entitas bernama Lagu_2511532014 di dalam package pekan8_2511532014.
3. Mendeklarasikan tiga atribut penampung informasi lagu, meliputi: String judul_2014 untuk menyimpan judul lagu, String penyanyi_2014 untuk nama penyanyi, dan int durasi_2014 untuk menyimpan panjang durasi lagu dalam satuan detik.
4. Membuat satu buah konstruktor bertipe publik yang menerima tiga parameter input untuk menginisialisasi nilai ketiga variabel atribut tersebut secara langsung saat objek lagu dibuat di memori.

b. Kode Program

```
1. package pekan8_2511532014;
2.
3. public class Lagu_2511532014 {
4.     String judul_2014;
5.     String penyanyi_2014;
6.     int durasi_2014; // dalam detik
7.
8.     public Lagu_2511532014(String judul_2014, String
penyanyi_2014, int durasi_2014) {
9.         this.judul_2014 = judul_2014;
10.        this.penyanyi_2014 = penyanyi_2014;
11.        this.durasi_2014 = durasi_2014;
12.    }
13.
14. }
15.
```

c. Analisis

Kelas Lagu_2511532014 bertindak sebagai kelas *Blueprint* atau struktur model data dasar (*Plain Old Java Object* / POJO). Desain kelas ini mengelompokkan data primitif dan teks majemuk (judul, penyanyi, durasi) menjadi satu kesatuan tipe data bentukan (*Object*). Berbeda dengan array satu dimensi biasa yang hanya bisa menyimpan satu tipe data seragam, objek dari kelas ini memungkinkan struktur data kompleks disimpan bersama di memori heap komputer. Atribut judul_2014 pada objek inilah yang nantinya menjadi kunci utama pembandingan alfabetis di dalam algoritma *Merge Sort* kelas penggerak.

2. Class Sorting_2511532014

a. Langkah Pengerjaan

1. Membuat kelas penggerak bernama Sorting_2511532014 yang mengelola array objek Lagu_2511532014[] dataLagu_2014 dengan batas kapasitas maksimal 20 elemen.
2. Mengembangkan fungsi inputData_2014() untuk mengisi array dengan data lagu awal secara hardcoded (seperti lagu Isyana Sarasvati, Via Vallen, Afgan, dll.) dan mencatat jumlah total lagu aktif ke dalam variabel jumlahLagu_2014.
3. Membuat fungsi tampilData_2014(String pesan_2014) menggunakan format teks System.out.printf untuk merender tampilan data tabel playlist secara rapi di konsol layar.
4. Mengimplementasikan metode merge_2014 khusus objek. Fungsi ini mengalokasikan dua array objek temporer L_2014[] dan R_2014[], menyalin data objek lagu ke dalamnya, kemudian membandingkan judul lagu secara alfabetis menggunakan perintah perbandingan string: L_2014[i_2014].judul_2014.compareToIgnoreCase(R_2014[j_2014].judul_2014) <= 0. Hasil penggabungan yang terurut dituliskan kembali ke array utama.
5. Mengimplementasikan fungsi rekursif mergeSort_2014(int l_2014, int r_2014) untuk membagi indeks jangkauan array secara rekursif berdasarkan titik tengah geometris (mid), lalu memanggil fungsi merge_2014 untuk menyatukan kembali sub-playlist.
6. Menyusun alur eksekusi pada metode main untuk menampilkan data playlist sebelum disorting, meminta konfirmasi interaksi pengguna melalui tombol Enter via Scanner, menjalankan pengurutan rekursif, dan menampilkan hasil akhir tabel playlist yang sudah terurut rapi alfabetis A-Z berdasarkan judul lagunya.

b. Kode Program

```
1. package pekan8_2511532014;
2.
3. import java.util.Scanner;
4.
5. public class Sorting_2511532014 {
6.     //array untuk menyimpan lagu max = 20
7.     static Lagu_2511532014[] dataLagu_2014 = new
Lagu_2511532014[20];
8.     static int jumlahLagu_2014 = 0;
9.
10.    // input data
11.    static void inputData_2014(){
12.
13.        dataLagu_2014[0] = new Lagu_2511532014("Awan Dan
Angin", "Isyana Sarasvati", 214);
14.        dataLagu_2014[1] = new Lagu_2511532014("Meraih
Bintang", "Via Vallen", 243);
```

```

15.     dataLagu_2014[2] = new Lagu_2511532014("Lagu Untukmu",
    "Afgan", 198);
16.     dataLagu_2014[3] = new Lagu_2511532014("Rasa Yang
    Tertinggal", "Raisa", 265);
17.     dataLagu_2014[4] = new Lagu_2511532014("Bukan Rayuan
    Gombal", "Judika", 237);
18.     dataLagu_2014[5] = new Lagu_2511532014("Hati Yang Kau
    Sakiti", "Rossa", 251);
19.     dataLagu_2014[6] = new Lagu_2511532014("Demi Nama
    Cintamu", "Zara Leola", 223);
20.     jumlahLagu_2014 = 7;
21.
22. }
23.
24.     static void mergeSort_2014(Lagu_2511532014[] arr_2014,
    int kiri_2014, int kanan_2014) {
25.         if (kiri_2014 < kanan_2014) {
26.             int tengah_2014 = (kiri_2014 + kanan_2014)
    / 2;
27.
28.             mergeSort_2014(arr_2014, kiri_2014,
    tengah_2014);
29.             mergeSort_2014(arr_2014, tengah_2014 + 1,
    kanan_2014);
30.
31.             merge_2014(arr_2014, kiri_2014,
    tengah_2014, kanan_2014);
32.         }
33.     }
34.
35.     static void merge_2014(Lagu_2511532014[] arr_2014, int
    kiri_2014, int tengah_2014, int kanan_2014) {
36.         int n1_2014 = tengah_2014 - kiri_2014 + 1;
37.         int n2_2014 = kanan_2014 - tengah_2014;
38.
39.         //array sementara
40.         Lagu_2511532014[] kiriArr_2014 = new
    Lagu_2511532014[n1_2014];
41.         Lagu_2511532014[] kananArr_2014 = new
    Lagu_2511532014[n2_2014];
42.
43.         for (int i_2014 = 0; i_2014 < n1_2014; i_2014++)
44.             kiriArr_2014[i_2014] = arr_2014[kiri_2014 +
    i_2014];
45.         for (int j_2014 = 0; j_2014 < n2_2014; j_2014++)
46.             kananArr_2014[j_2014] =
    arr_2014[tengah_2014 + 1 + j_2014];
47.
48.         int i_2014 = 0, j_2014 = 0, k_2014 = kiri_2014;
49.
50.         while (i_2014 < n1_2014 && j_2014 < n2_2014) {
51.             if
    (kiriArr_2014[i_2014].judul_2014.compareToIgnoreCase(kananArr_2
    014[j_2014].judul_2014) <= 0) {
52.                 arr_2014[k_2014++] =
    kiriArr_2014[i_2014++];
53.             } else {

```

```

54.         arr_2014[k_2014++] =
        kananArr_2014[j_2014++];
55.     }
56. }
57.
58.     while (i_2014 < n1_2014)
59.         arr_2014[k_2014++] =
        kiriArr_2014[i_2014++];
60.     while (j_2014 < n2_2014)
61.         arr_2014[k_2014++] =
        kananArr_2014[j_2014++];
62. }
63.
64.     static void tampilData_2014(String label_2014) {
65.         System.out.println("\n" + label_2014);
66.         System.out.println("-".repeat(55));
67.
68.         for (int i_2014 = 0; i_2014 < jumlahLagu_2014;
        i_2014++) {
69.             Lagu_2511532014 l_2014 = dataLagu_2014[i_2014];
70.
71.             System.out.printf("%2d. %-35s - %d detik%n",
72.                 i_2014 + 1,
73.                 l_2014.judul_2014 + " (" +
        l_2014.penyanyi_2014 + ")",
74.                 l_2014.durasi_2014);
75.         }
76.
77.         System.out.println("-".repeat(55));
78.     }
79.
80.     public static void main(String[] args) {
81.         Scanner sc = new Scanner(System.in);
82.
83.         System.out.println("=".repeat(55));
84.         System.out.println("    SORTING PLAYLIST - NIM:
        2511532014");
85.         System.out.println("    Algoritma: Merge Sort (Judul A-
        Z)");
86.         System.out.println("=".repeat(55));
87.
88.         // Isi data awal
89.         inputData_2014();
90.
91.         // Tampilkan data sebelum sorting
92.         tampilData_2014("Data Sebelum Merge Sort:");
93.
94.         // Konfirmasi jalankan sorting
95.         System.out.print("\nJalankan Merge Sort? (y/n): ");
96.         String pilihan = sc.nextLine().trim().toLowerCase();
97.
98.         if (pilihan.equals("y")) {
99.             // Jalankan Merge Sort
100.            mergeSort_2014(dataLagu_2014, 0,
                jumlahLagu_2014 - 1);
101.
102.            // Tampilkan data sesudah sorting

```



```

103.         tampilData_2014("Data Setelah Merge Sort
        (Judul A-Z):");
104.         System.out.println("\nSorting selesai! Data
        telah diurutkan berdasarkan Judul A-Z.");
105.     } else {
106.         System.out.println("Sorting dibatalkan.");
107.     }
108.
109.     sc.close();
110. }
111.
112. }
113.

```

c. Output

```

<terminated> Sorting_2511532014 [Java Application] C:\Users\asus\p2\pool\plugins
=====
  SORTING PLAYLIST - NIM: 2511532014
  Algoritma: Merge Sort (Judul A-Z)
=====

Data Sebelum Merge Sort:
-----
  1. Awan Dan Angin (Isyana Sarasvati)    - 214 detik
  2. Meraih Bintang (Via Vallen)          - 243 detik
  3. Lagu Untukmu (Afgan)                 - 198 detik
  4. Rasa Yang Tertinggal (Raisa)         - 265 detik
  5. Bukan Rayuan Gombal (Judika)         - 237 detik
  6. Hati Yang Kau Sakiti (Rossa)         - 251 detik
  7. Demi Nama Cintamu (Zara Leola)       - 223 detik
-----

Jalankan Merge Sort? (y/n): y

Data Setelah Merge Sort (Judul A-Z):
-----
  1. Awan Dan Angin (Isyana Sarasvati)    - 214 detik
  2. Bukan Rayuan Gombal (Judika)         - 237 detik
  3. Demi Nama Cintamu (Zara Leola)       - 223 detik
  4. Hati Yang Kau Sakiti (Rossa)         - 251 detik
  5. Lagu Untukmu (Afgan)                 - 198 detik
  6. Meraih Bintang (Via Vallen)          - 243 detik
  7. Rasa Yang Tertinggal (Raisa)         - 265 detik
-----

Sorting selesai! Data telah diurutkan berdasarkan Judul A-Z.

```

d. Analisis

Program ini mengintegrasikan struktur data objek ke dalam algoritma pengurutan tingkat lanjut (Advanced Sorting). Analisis pada fungsionalitas perbandingan memperlihatkan adaptasi logika: jika pengurutan array bilangan integer (pada sub-bab sebelumnya) menggunakan operator relasional matematika ($\leq$), maka pada objek penataan lagu ini sistem wajib membandingkan variabel teks menggunakan metode `compareToIgnoreCase()`. Fungsi ini mengembalikan nilai bilangan negatif atau nol jika judul lagu pada sub-array kiri secara alfabetis harus mendahului judul lagu di sub-array kanan.

Penerapan Merge Sort di sini menjamin performa pengurutan playlist musik yang sangat stabil dengan kompleksitas waktu konstan sebesar $O(n \log n)$ dalam kondisi apa pun. Hal ini sangat menguntungkan untuk aplikasi pemutar musik nyata, karena efisiensinya tidak dipengaruhi oleh seberapa acak urutan lagu awal di dalam daftar putar. Kelemahan dari alur penggabungan ini adalah ia menciptakan duplikasi alamat referensi objek baru pada array penampung pembantu `L_2014[]` dan `R_2014[]` di dalam memori heap komputer pada setiap kali fase penggabungan (`merge_2014`) dipanggil oleh rantai fungsi rekursif. Namun, hal ini dibayar dengan hasil penataan playlist lagu yang rapi dan terurut stabil secara alfabetis (A-Z).